Dockerized Healthcare Python Flask Service Report

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Animesh Bokil	EN22CS301125
02	Anshika Vyas	EN22CS301154
03	Aryan Singh	EN22CS301215
04	Ashlesha Verma	EN22CS301219
05	Avani Rawal	EN22CS301238

Group Name: 10D2

Project Number: DO-22

Industry Mentor Name: Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

Table of Contents

1. **Problem Statement s Objectives**
   - 1.1 Problem Statement
   - 1.2 Project Objectives
   - 1.3 Scope of the Project
2. **Proposed Solution**
   - 2.1 Key Features
   - 2.2 Overall Architecture / Workflow
   - 2.3 Tools C Technologies Used
3. **Results s Output**
   - 3.1 Screenshots / Outputs
   - 3.2 Reports / Dashboards
   - 3.3 Key Outcomes
4. **Conclusion**
5. **Future Scope s Enhancements**
6. **References**
7. **GitHub**

1. Problem Statement s Objectives

1.1 Problem Statement

Modern healthcare applications require a reliable, scalable, and consistent deployment environment to manage patient records, appointments, and medical data efficiently. However, traditional deployment methods often lead to dependency conflicts, environment inconsistencies, and difficulties in scaling.The Healthcare Python Flask Service needed a production-ready solution that could ensure consistent execution across environments, automate deployment, and support scalability. Additionally, there was a requirement to integrate containerization and orchestration technologies to simplify development, testing, and production workflows

1.2 Project Objectives

- Containerize the Healthcare Flask application using Docker with best practices
- Implement multi-stage Docker builds to optimize image size and improve security
- Set up Docker Compose for multi-container orchestration (Flask + Database)
- Build a CI/CD pipeline using GitHub Actions/Jenkins for automation
- Deploy the application on AWS EC2 / Kubernetes cluster
- Ensure scalability, reliability, and consistency across environments
- Manage configuration and secrets securely using environment variables

1.3 Scope of the Project

This project focuses on designing and implementing a complete DevOps pipeline for a healthcare application. It includes Docker-based containerization, multi-container orchestration using Docker Compose, CI/CD pipeline automation, and deployment on cloud infrastructure.

The project covers infrastructure, deployment, and DevOps practices but does not focus on advanced healthcare analytics or AI-based diagnosis systems. The emphasis is on building a scalable and portable deployment architecture.

2. Proposed Solution

2.1 Key Features

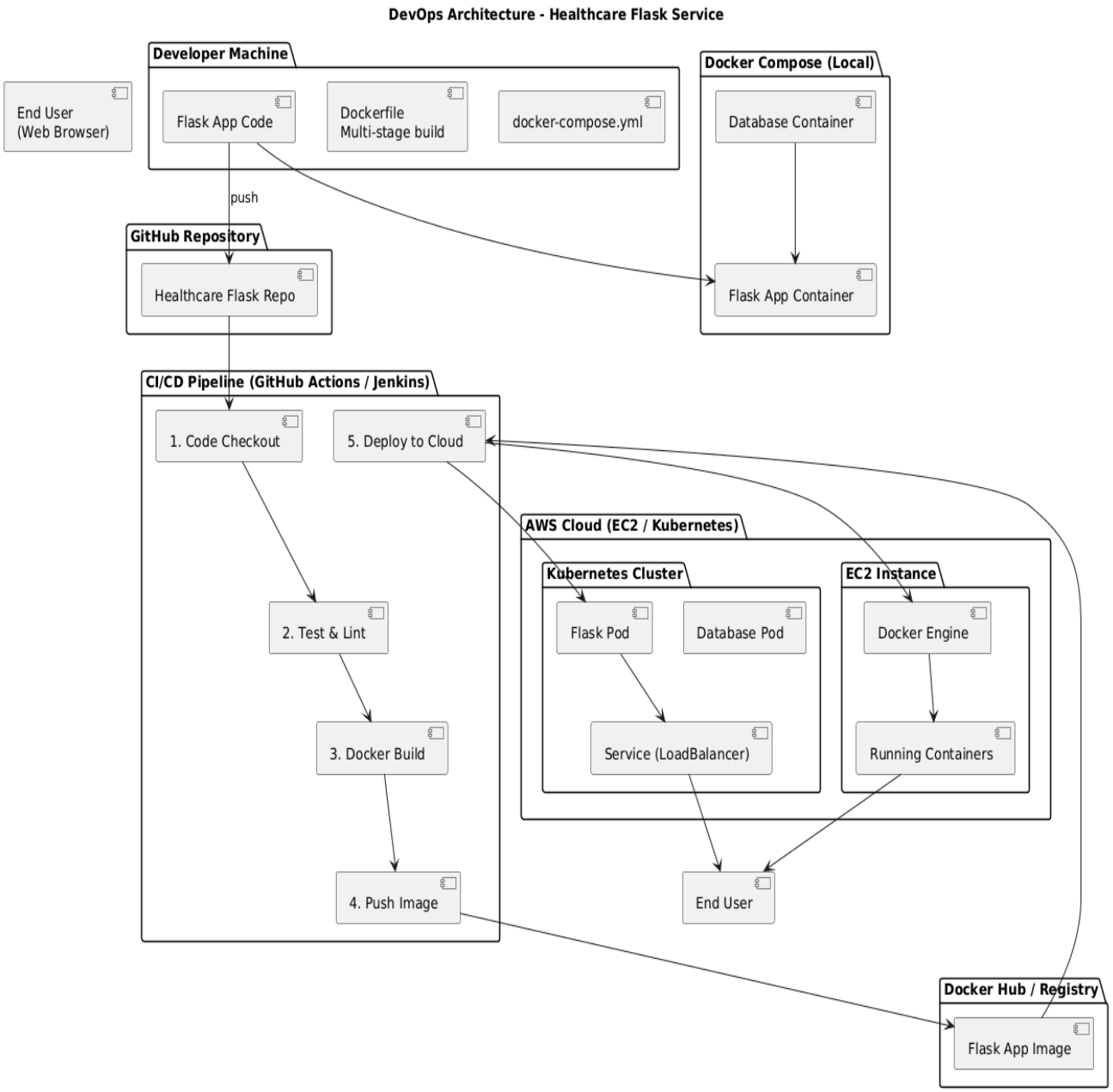
- Containerized Flask-based healthcare service for consistent deployment
- Multi-stage Dockerfile for optimized, lightweight, and secure images
- Docker Compose for managing Flask app and database services
- Automated CI/CD pipeline with build, test, and deployment stages
- Cloud deployment using AWS EC2 / Kubernetes
- Secure configuration using environment variables
- Modular and scalable architecture

2.2 Overall Architecture / Workflow

The system follows a DevOps-based workflow:

1. **Developer writes and updates Flask application code**
2. **Code is pushed to GitHub repository**
3. **CI pipeline is triggered automatically**
4. **Code is tested, linted, and validated**
5. **Docker image is built using a multi-stage Dockerfile**
6. **Image is pushed to Docker Hub (or registry)**
7. **Docker Compose runs services locally (Flask + Database)**
8. **CD pipeline deploys the application to AWS EC2 / Kubernetes**
9. **Application becomes accessible to users**

This workflow ensures automation, consistency, and faster delivery.

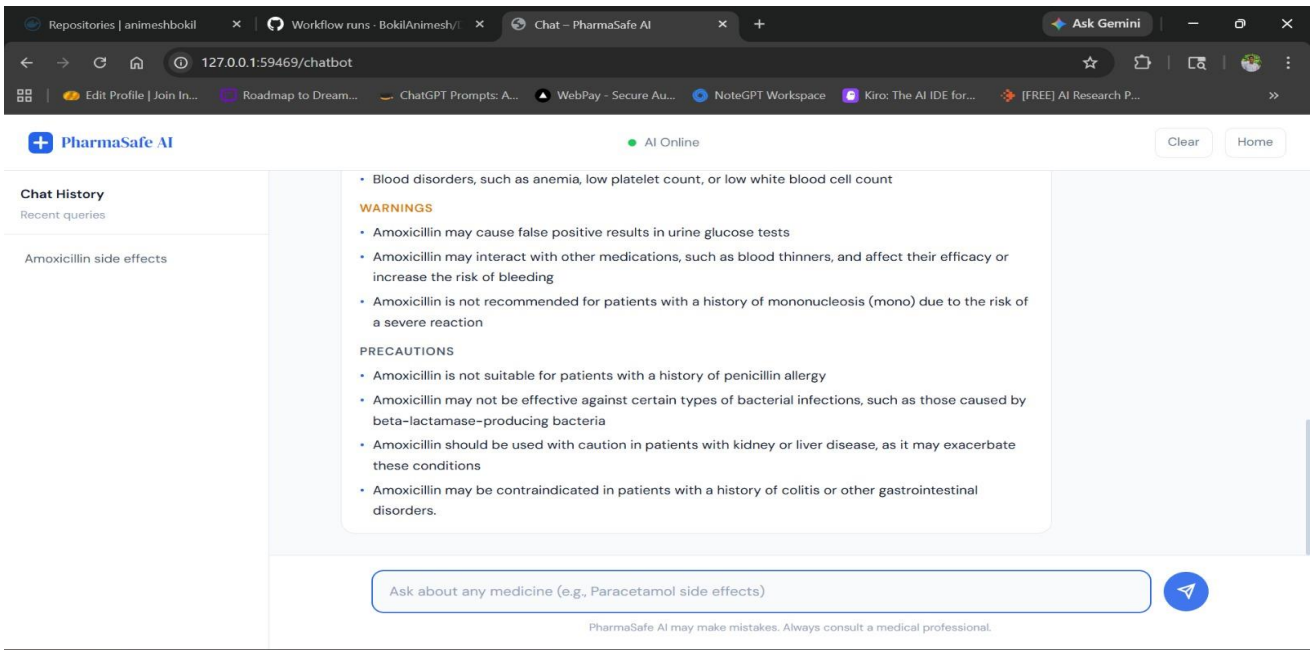

DevOps Architecture - Healthcare Flask Service
Developer Machine
End User
(Web Browser)
Flask App Code
Dockerfile
Multi-stage build
docker-compose.yml
Docker Compose (Local)
Database Container
Flask App Container
push
GitHub Repository
Healthcare Flask Repo
CI/CD Pipeline (GitHub Actions / Jenkins)
1. Code Checkout
5. Deploy to Cloud
2. Test & Lint
3. Docker Build
4. Push Image
AWS Cloud (EC2 / Kubernetes)
Kubernetes Cluster
Flask Pod
Database Pod
Service (LoadBalancer)
EC2 Instance
Docker Engine
Running Containers
End User
Docker Hub / Registry
Flask App Image

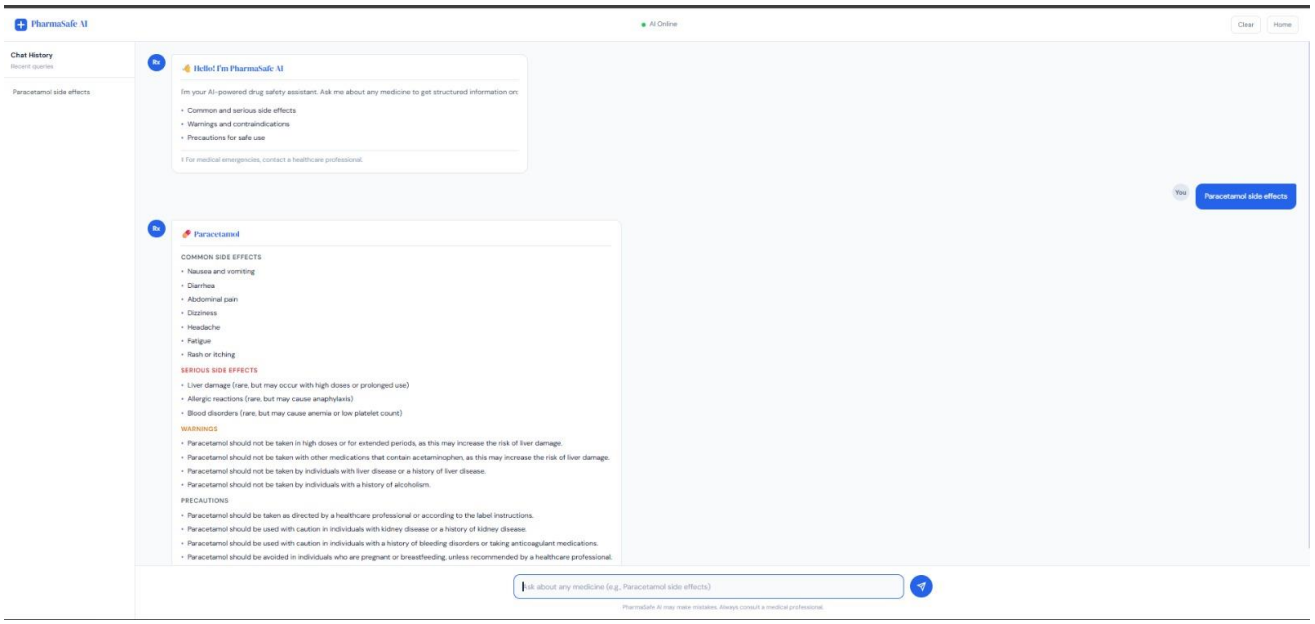
2.3 Tools s Technologies Used

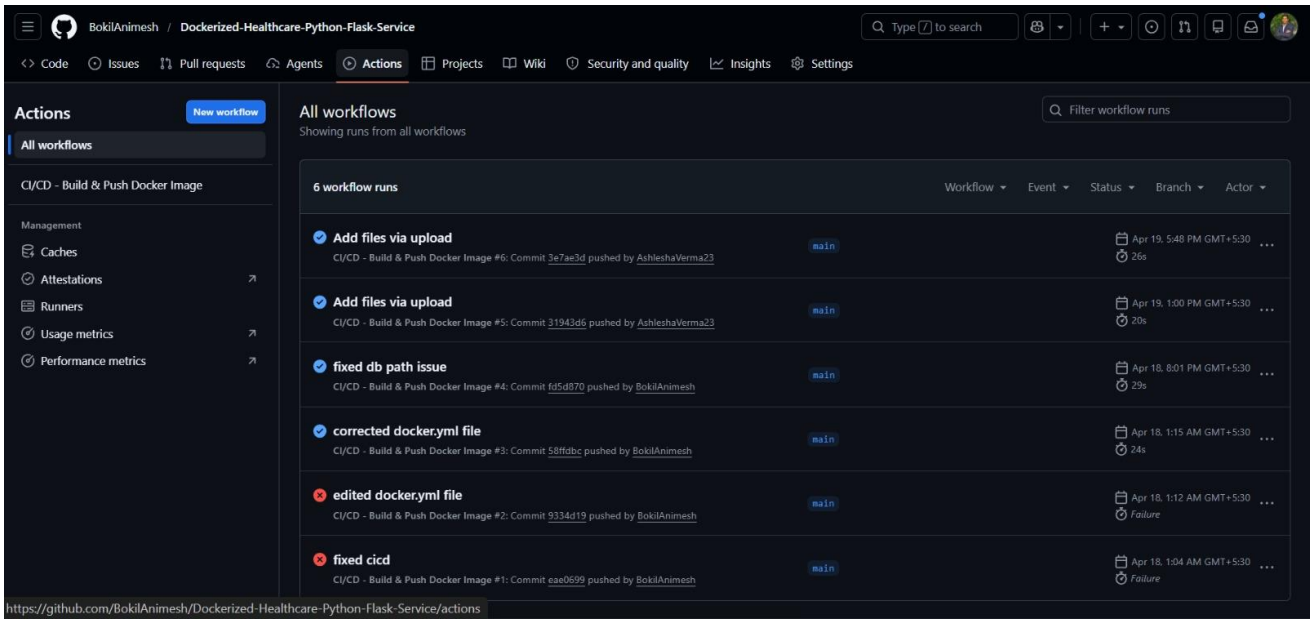
Category	Tool / Technology
Programming	Python, Flask
Containerization	Docker, Docker Compose
Orchestration	Kubernetes (K8s)
CI/CD	GitHub Actions / Jenkins
Cloud	AWS EC2
Database	PostgreSQL / MySQL
Version Control	Git, GitHub
Monitoring	Docker Logs

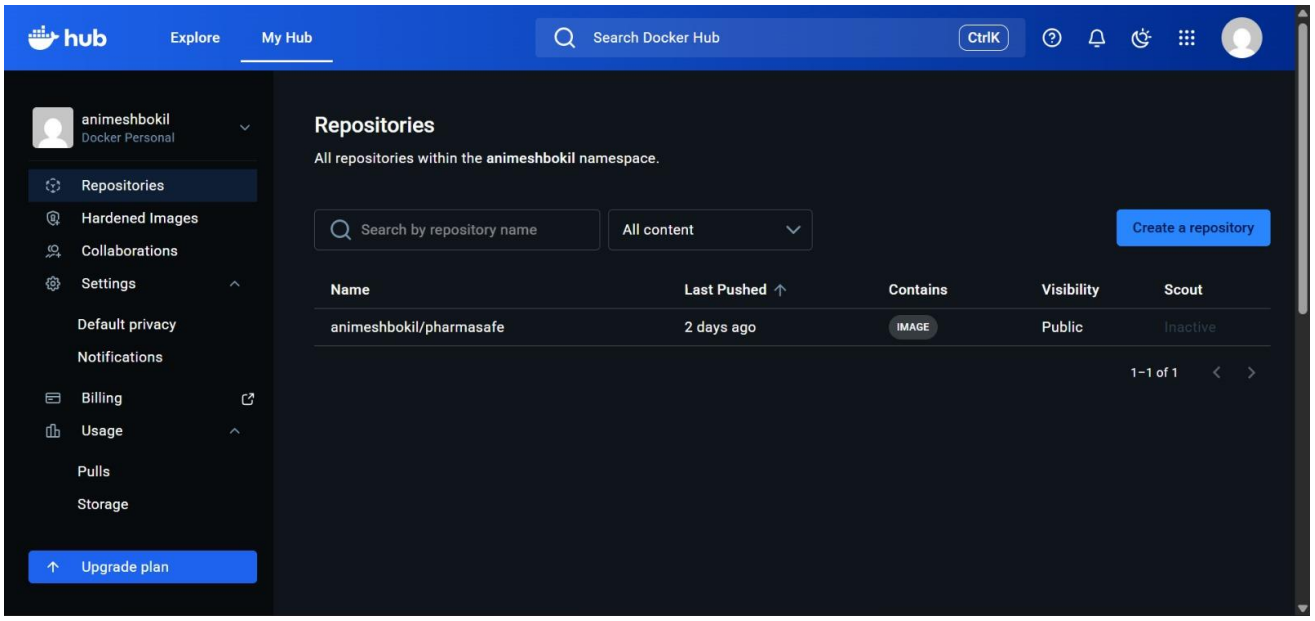
3. Results/Output

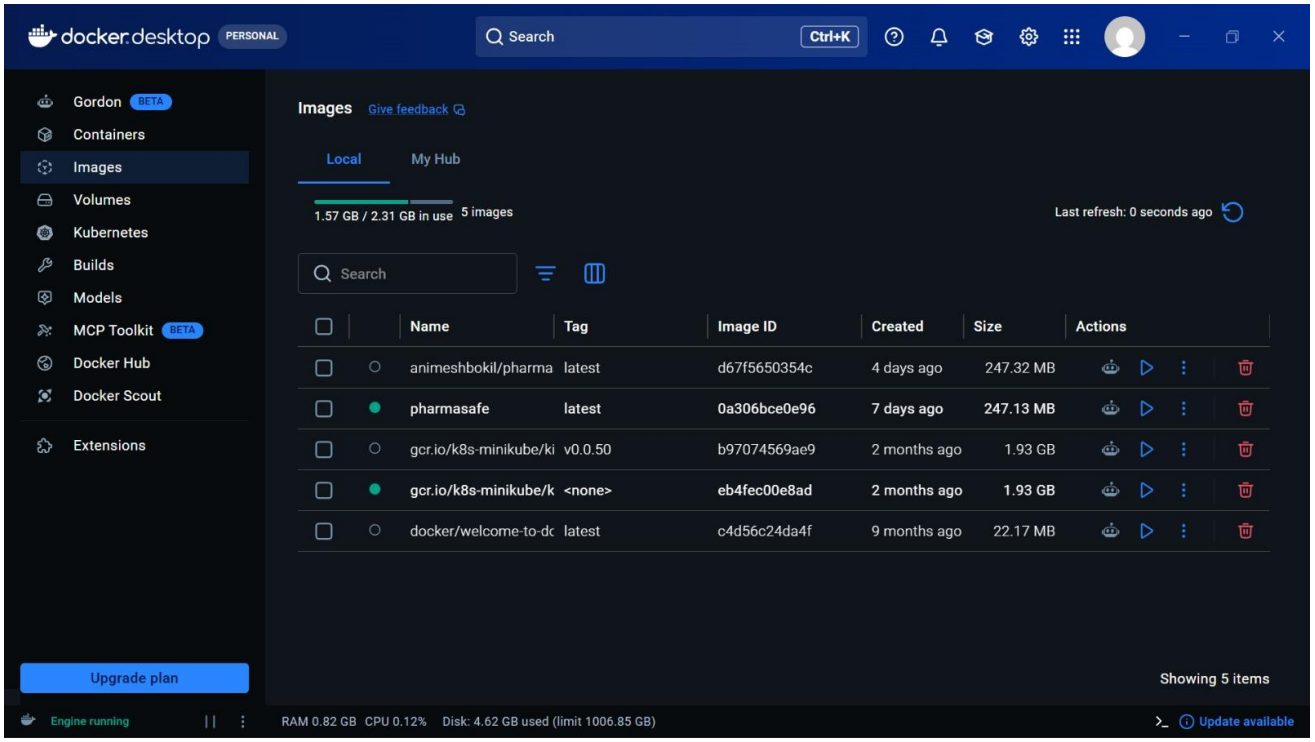
3.1 Screenshots / Outputs




PharmaSafe AI
AI Online
Clear
Home
Chat History
Recent queries
Paracetamol side effects
Hello! I'm PharmaSafe AI
I'm your AI-powered drug safety assistant. Ask me about any medicine to get structured information on:
• Common and serious side effects
• Warnings and contraindications
• Precautions for safe use
For medical emergencies, contact a healthcare professional.
You
Paracetamol side effects
Paracetamol
COMMON SIDE EFFECTS
• Nausea and vomiting
• Diarrhea
• Abdominal pain
• Dizziness
• Headache
• Fatigue
• Rash or itching
SERIOUS SIDE EFFECTS
• Liver damage (rare, but may occur with high doses or prolonged use)
• Allergic reactions (rare, but may cause anaphylaxis)
• Blood disorders (rare, but may cause anemia or low platelet count)
WARNINGS
• Paracetamol should not be taken in high doses or for extended periods, as this may increase the risk of liver damage.
• Paracetamol should not be taken with other medications that contain acetaminophen, as this may increase the risk of liver damage.
• Paracetamol should not be taken by individuals with liver disease or a history of liver disease.
• Paracetamol should not be taken by individuals with a history of alcoholism.
PRECAUTIONS
• Paracetamol should be taken as directed by a healthcare professional or according to the label instructions.
• Paracetamol should be used with caution in individuals with kidney disease or a history of kidney disease.
• Paracetamol should be used with caution in individuals with a history of bleeding disorders or taking anticoagulant medications.
• Paracetamol should be avoided in individuals who are pregnant or breastfeeding, unless recommended by a healthcare professional.
Ask about any medicine (e.g., Paracetamol side effects)
PharmaSafe AI may make mistakes. Always consult a medical professional.


BokilAnimesh / Dockerized-Healthcare-Python-Flask-Service
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Actions
New workflow
All workflows
CI/CD - Build & Push Docker Image
Management
Caches
Attestations
Runners
Usage metrics
Performance metrics
All workflows
Showing runs from all workflows
Filter workflow runs
6 workflow runs
Workflow
Event
Status
Branch
Actor
Add files via upload
CI/CD - Build & Push Docker Image #6: Commit 3e7ae3d pushed by AshleshaVerma23
main
Apr 19, 5:48 PM GMT+5:30
26s
Add files via upload
CI/CD - Build & Push Docker Image #5: Commit 31943d6 pushed by AshleshaVerma23
main
Apr 19, 1:00 PM GMT+5:30
20s
fixed db path issue
CI/CD - Build & Push Docker Image #4: Commit fd5d870 pushed by BokilAnimesh
main
Apr 18, 8:01 PM GMT+5:30
29s
corrected docker.yml file
CI/CD - Build & Push Docker Image #3: Commit 58ffdbc pushed by BokilAnimesh
main
Apr 18, 1:15 AM GMT+5:30
24s
edited docker.yml file
CI/CD - Build & Push Docker Image #2: Commit 9334d19 pushed by BokilAnimesh
main
Apr 18, 1:12 AM GMT+5:30
Failure
fixed cicd
CI/CD - Build & Push Docker Image #1: Commit eae0699 pushed by BokilAnimesh
main
Apr 18, 1:04 AM GMT+5:30
Failure
https://github.com/BokilAnimesh/Dockerized-Healthcare-Python-Flask-Service/actions


hub
Explore
My Hub
Search Docker Hub
CtrlK
animeshbokil
Docker Personal
Repositories
Hardened Images
Collaborations
Settings
Default privacy
Notifications
Billing
Usage
Pulls
Storage
Upgrade plan
Repositories
All repositories within the animeshbokil namespace.
Search by repository name
All content
Create a repository
Name
Last Pushed
Contains
Visibility
Scout
animeshbokil/pharmasafe
2 days ago
IMAGE
Public
Inactive
1–1 of 1


docker desktop
PERSONAL
Search
Ctrl+K
Gordon
BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit
BETA
Docker Hub
Docker Scout
Extensions
Upgrade plan
Images
Give feedback
Local
My Hub
1.57 GB / 2.31 GB in use
5 images
Last refresh: 0 seconds ago
Search
Name
Tag
Image ID
Created
Size
Actions
animeshbokil/pharma
latest
d67f5650354c
4 days ago
247.32 MB
pharmasafe
latest
0a306bce0e96
7 days ago
247.13 MB
gcr.io/k8s-minikube/ki
v0.0.50
b97074569ae9
2 months ago
1.93 GB
gcr.io/k8s-minikube/k
<none>
eb4fec00e8ad
2 months ago
1.93 GB
docker/welcome-to-dc
latest
c4d56c24da4f
9 months ago
22.17 MB
Showing 5 items
Engine running
RAM 0.82 GB CPU 0.12% Disk: 4.62 GB used (limit 1006.85 GB)
Update available

3.2 Reports / Dashboards / Models

Docker Container Status:

- Flask Application → Running
- Database Service → Running

Docker Compose Services:

- flask-app
- database

CI/CD Pipeline Stages:

Stage	Status
Code Checkout	Passed
Linting	Passed
Testing	Passed
Docker Build	Passed
Docker Push	Passed
Deployment	Passed

3.3 Key Outcomes

- Successfully containerized the healthcare application using Docker
- Achieved consistent deployment across different environments
- Implemented CI/CD pipeline for automated build and deployment
- Reduced image size using multi-stage Docker builds
- Simplified multi-service management using Docker Compose
- Enabled scalability and portability of the application
- Improved development workflow and team collaboration

4. Conclusion

This project demonstrates a complete DevOps implementation for a Healthcare Python Flask Service. By integrating Docker, Docker Compose, CI/CD pipelines, and cloud deployment, we achieved a scalable and efficient deployment system.

The key learning from this project is understanding how containerization, automation, and orchestration work together to create a reliable software delivery pipeline. The use of multi-stage builds, automated testing, and cloud deployment ensures high performance and maintainability.

Overall, the project successfully transforms a simple Flask application into a production-ready, scalable healthcare service.

5. Future Scopes Enhancements

- Implement Kubernetes auto-scaling using Horizontal Pod Autoscaler
- Add Redis caching for faster performance
- Integrate monitoring tools like Prometheus and Grafana
- Enable HTTPS using SSL certificates
- Add authentication and authorization for secure access
- Deploy using AWS EKS for fully managed Kubernetes
- Introduce microservices architecture for modular expansion

6. References

- Docker Documentation – https://docs.docker.com
- Flask Documentation – https://flask.palletsprojects.com
- Kubernetes Documentation – https://kubernetes.io/docs
- GitHub Actions Documentation – https://docs.github.com/actions
- AWS EC2 Documentation – https://docs.aws.amazon.com/ec2

7. GitHub

https://github.com/BokilAnimesh/Dockerized-Healthcare-Python-Flask-Service.git